

Docker Basics: Install Docker and Run a Web Application on Ubuntu EC2

Task Description

Objective

To understand the **basic concept of Docker**, including Docker installation, running containers, port mapping, and creating a simple Docker image using an **Ubuntu EC2 instance**.

Task Steps

Step 1: Launch EC2 Instance

- Create an **Ubuntu EC2 instance**.
- Connect to the instance using **SSH (Command Prompt / Terminal)**.

Step 2: Install Docker on Ubuntu

Step 3: Run NGINX Container

- Run an NGINX container using Docker.
- Expose port **80** using Docker port mapping.
- Update **EC2 Security Group** to allow inbound HTTP (port 80).
- Access NGINX using the **EC2 public IP**.

Example:

```
docker run -d -p 80:80 nginx
```

Step 4: Work with Docker Containers

- List running and stopped containers.
- Inspect a running container.
- Understand container status.

Commands:

```
docker ps -a  
docker inspect <container-id>
```

Step 5: Create and Use Docker Image

- Run an Ubuntu container.
- Enter into the container using interactive mode.
- Make a small change (example: update a file).
- Create a Docker image from the container.
- Push the image to **Docker Hub**.

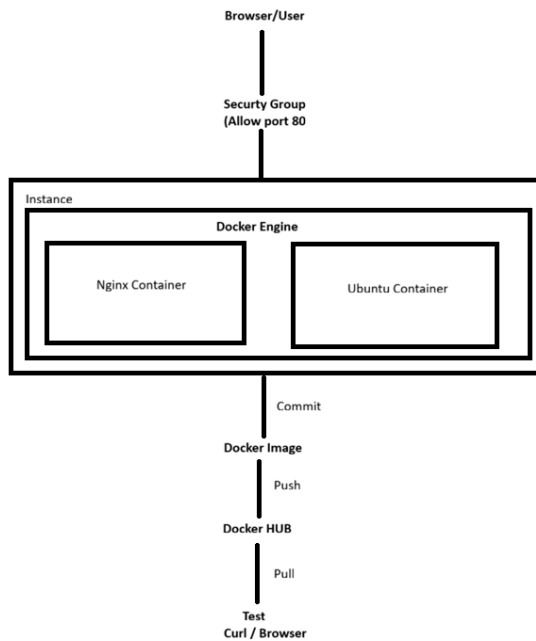
Step 6: Verify the Application

- Start a container from the created image.
- Verify the output using curl or browser.

Expected Outcome

- Docker successfully installed on Ubuntu.
- NGINX running inside a Docker container.
- Application accessible using EC2 public IP.
- Docker image created and pushed to Docker Hub.
- Basic understanding of Docker images and containers.

Solution :

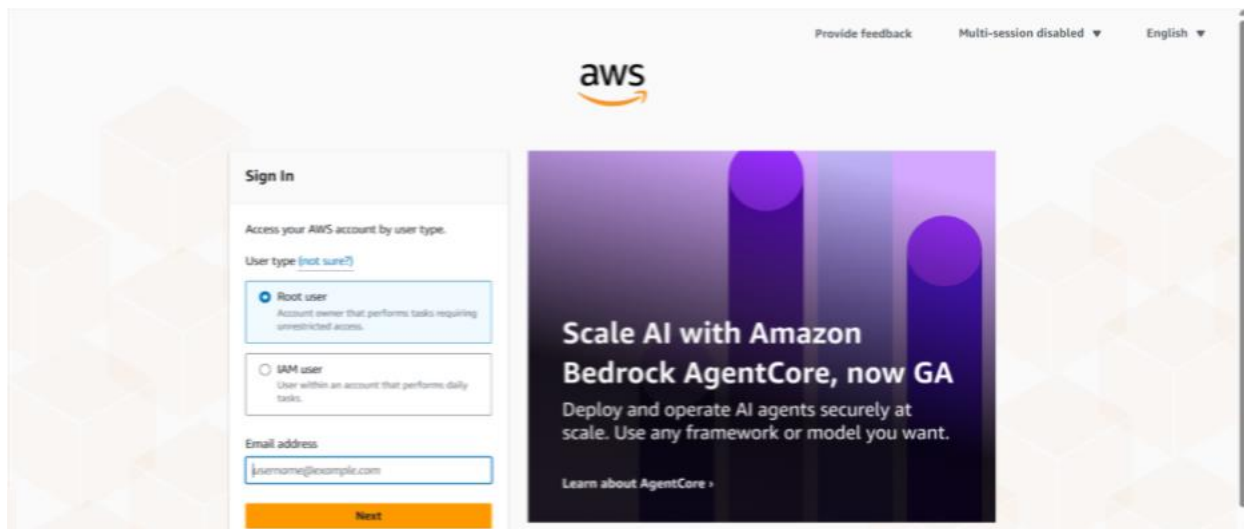


Work flow:

Login to your AWS account with credentials and required verification.

1. (<https://signin.aws.amazon.com/>)

My account – AWS Management Console- Sign in



2. Launch an Ubuntu EC2 Instance

- Go to AWS Console → EC2 → Launch Instance
- AMI: Ubuntu Server 22.04 LTS
- Instance type: t2.micro (Free Tier)
- key pair

- Security Group:
 - Allow SSH (22) from *your IP*
 - add HTTP (80) later
- Launch

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 22.04 LTS (HVM), SSD Volume Type

ami-0c398cb65a93047f2 (64-bit (x86)) / ami-0f14ad9f1d341c53d (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Ubuntu Server 22.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 22.04, amd64 jammy image

Architecture

AMI ID

Publish Date

Username

Verified provider

64-bit (x86)

ami-0c398cb65a93047f2

2025-10-15

ubuntu

Software Image (AMI)

Canonical, Ubuntu, 22.04, amd6...[read more](#)

ami-0c398cb65a93047f2

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

[Preview code](#)

▼ Instance type

Info | [Get advice](#)

Instance type

t2.micro

Family: t2 1 vCPU 1 GiB Memory Current generation: true

On-Demand Windows base pricing: 0.0162 USD per Hour On-Demand SUSE base pricing: 0.0116 USD per Hour On-Demand RHEL base pricing: 0.026 USD per Hour On-Demand Linux base pricing: 0.0116 USD per Hour

All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)

Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

demo

[Create new key pair](#)

▼ Network settings

Info

Network

Info

Network settings

Info

Network

Info

Subnet

Info

No preference (Default subnet in any availability zone)

Auto-assign public IP

Info

Enable

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-2' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

My IP

110.227.211.247/32

☐ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☐ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

▼ Summary

Number of instances

Info

1

Software Image (AMI)

Canonical, Ubuntu, 22.04, amd6...[read more](#)

ami-0c398cb65a93047f2

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

[Preview code](#)

3. Connect to EC2 using SSH

```

* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:      https://ubuntu.com/pro

System information as of Mon Jan 5 06:17:36 UTC 2026

System load: 0.44          Processes: 103
Usage of /: 30.9% of 7.57GB Users logged in: 0
Memory usage: 26%         IPv4 address for eth0: 172.31.26.69
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

updates can be applied immediately.

enable ESM Apps to receive additional future security updates.
see https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

** System restart required **
Last login: Mon Jan 5 06:12:34 2026 from 18.206.107.29
ubuntu@ip-172-31-26-69:~$

```

```

Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-apt-common all 2.4.0ubuntu4.1 [14.7 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-apt amd64 2.4.0ubuntu4.1 [164 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-pro-client-l10n amd64 37.1ubuntu0-22.04 [20.7 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-pro-client amd64 37.1ubuntu0-22.04 [238 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-advantage-tools all 37.1ubuntu0-22.04 [10.9 kB]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 apparmor amd64 3.0.4-2ubuntu2.5 [599 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 bind9-dnsutils amd64 1:9.18.39-0ubuntu0.22.04.2 [158 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 bind9-host amd64 1:9.18.39-0ubuntu0.22.04.2 [52.5 kB]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 bind9-libs amd64 1:9.18.39-0ubuntu0.22.04.2 [1262 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpq16-16 amd64 1.6.37-3ubuntu0.1 [191 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libcrt0 amd64 2.38-4ubuntu2.12 [103 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libcrt-nobfd0 amd64 2.38-4ubuntu2.12 [108 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 binutils-x86-64-linux-gnu amd64 2.38-4ubuntu2.12 [2324 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libbinutils amd64 2.38-4ubuntu2.12 [663 kB]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 binutils amd64 2.38-4ubuntu2.12 [3184 B]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 binutils-common amd64 2.38-4ubuntu2.12 [223 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 landscape-common amd64 23.02-0ubuntu1-22.04.7 [88.9 kB]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libssh-4 amd64 0.9.6-2ubuntu0.22.04.5 [187 kB]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-modules-6.8.0-1044-aws amd64 6.8.0-1044.46-22.04.1 [25.3 MB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-image-6.8.0-1044-aws amd64 6.8.0-1044.46-22.04.1 [14.7 MB]
Get:29 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-aws amd64 6.8.0-1044.46-22.04.1 [1718 B]
Get:30 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-image-aws amd64 6.8.0-1044.46-22.04.1 [2458 B]
Get:31 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-aws-6.8-headers-6.8.0-1044 all 6.8.0-1044.46-22.04.1 [13.5 MB]
Get:32 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers-6.8.0-1044-aws amd64 6.8.0-1044.46-22.04.1 [3504 kB]
Get:33 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers-aws amd64 6.8.0-1044.46-22.04.1 [2372 B]
Get:34 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-tools-common all 5.15.0-164.174 [296 kB]
Get:35 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-aws-6.8-tools-6.8.0-1044 amd64 6.8.0-1044.46-22.04.1 [3588 kB]
Get:36 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-tools-6.8.0-1044-aws amd64 6.8.0-1044.46-22.04.1 [1792 B]
Get:37 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-urllib3 all 1.26.5-1-explubuntu0.4 [98.6 kB]
Get:38 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 snapd amd64 2.72+ubuntu22.04 [31.8 MB]
Get:39 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 sosreport amd64 4.9.2-0ubuntu0-22.04.1 [367 kB]
Get:40 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 intel-microcode amd64 3.20250812.0ubuntu0.22.04.1 [11.5 MB]
Get:41 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 intel-microcode amd64 3.20250812.0ubuntu0.22.04.1 [11.5 MB]
Fetched 119 MB in 2s (77.5 MB/s)

```

Install Docker o Ubuntu

- *sudo apt install docker.io -y*
- *sudo systemctl start docker*
- *sudo systemctl enable docker*
- *sudo systemctl status docker*

```

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-26-69:~$ sudo systemctl start docker
sudo systemctl enable docker
ubuntu@ip-172-31-26-69:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-01-05 06:18:56 UTC; 43s ago
 TriggeredBy: ● docker.socket
   Main PID: 13782 (dockerd)
     Tasks: 8
    Memory: 29.0M
       CPU: 300ms
   CGroup: /system.slice/docker.service
           └─13782 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jan 05 06:18:55 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:55.626841516Z" level=info msg="Loading containers: start."
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.036581626Z" level=info msg="Loading containers: done."
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.057087584Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1-22.04.1" containerd-snapshott
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.057186172Z" level=info msg="Initializing buildkit"
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.061539352Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.061558053Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.085914043Z" level=info msg="Completed buildkit initialization"
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.095806277Z" level=info msg="Daemon has completed initialization"
Jan 05 06:18:56 ip-172-31-26-69 dockerd[13782]: time="2026-01-05T06:18:56.095936316Z" level=info msg="API listen on /run/docker.sock"

```

Run Docker without sudo

sudo usermod -aG docker ubuntu

Run this cmd then log out and log In again.

After check its working or not

docker ps

```

ubuntu@ip-172-31-26-69:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-26-69:~$

```

4. Run NGINX Container

Run following cmd to run Nginx using Docker

docker run -d -p 80:80 --name mynginx nginx

```

ubuntu@ip-172-31-26-69:~$ docker run -d -p 80:80 --name mynginx nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
02d7611c4eae: Pull complete
dcea87ab9c4a: Pull complete
35df28ad1026: Pull complete
99ae2d6d05ef: Pull complete
a2b008488679: Pull complete
d03ca78f31fe: Pull complete
d6799cf0ce70: Pull complete
Digest: sha256:ca871a86d45a3ec6864dc45f014b11fe626145569ef0e74deaffc95a3b15b430
Status: Downloaded newer image for nginx:latest
a889fbf9204a7d698decc8b8ea382dcabc0f3c1160e3391ed0e206a0bba97bc8b
ubuntu@ip-172-31-26-69:~$

```

Verify container image is running

```
ubuntu@ip-172-31-26-69:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
a889fbf9204a   nginx    "/docker-entrypoint...." 41 seconds ago Up 41 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp mynginx
ubuntu@ip-172-31-26-69:~$
```

5. Update EC2 Security Group

- EC2 - Security Groups - Inbound Rules - Edit
- Add:
 - Type: HTTP
 - Port: 80
 - Source: 0.0.0.0/0

6. Access NGINX in browser

http:<public ip>



7. Create and use Docker Image.

```
docker run -it --name myubuntu ubuntu bash
```

Run this cmd to get into the docker container

```
ubuntu@ip-172-31-26-69:~$ docker run -it --name myubuntu ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
20043066d3d5: Pull complete
Digest: sha256:c35e29c9450151419d9448b0fd75374fec4fff364a27f176fb458d472dfc9e54
Status: Downloaded newer image for ubuntu:latest
root@72c4f6075caf:/#
```

7.1 Make changes.

Create a file with a message in it

To check that our container is running properly or not

```
apt update
```

```
echo "Hello from my custom Docker image" > /myfile.txt
```

```
cat /myfile.txt
```

```
root@72c4f6075caf:/# apt update
echo "Hello from my custom Docker image" > /myfile.txt
cat /myfile.txt
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [33.1 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1752 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2898 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1183 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:11 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1950 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [35.9 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2130 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3059 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [49.5 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [34.6 kB]
Fetched 35.3 MB in 4s (8504 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
9 packages can be upgraded. Run 'apt list --upgradable' to see them.
Hello from my custom Docker image
root@72c4f6075caf:/#
```

Then exit from container

```
exit
```

7.2 Create Docker image from container

```
docker commit myubuntu mydockerhubusername/myubuntuimage:v1
```

This cmd will create a docker image

docker images

This cmd will show the created image of docker

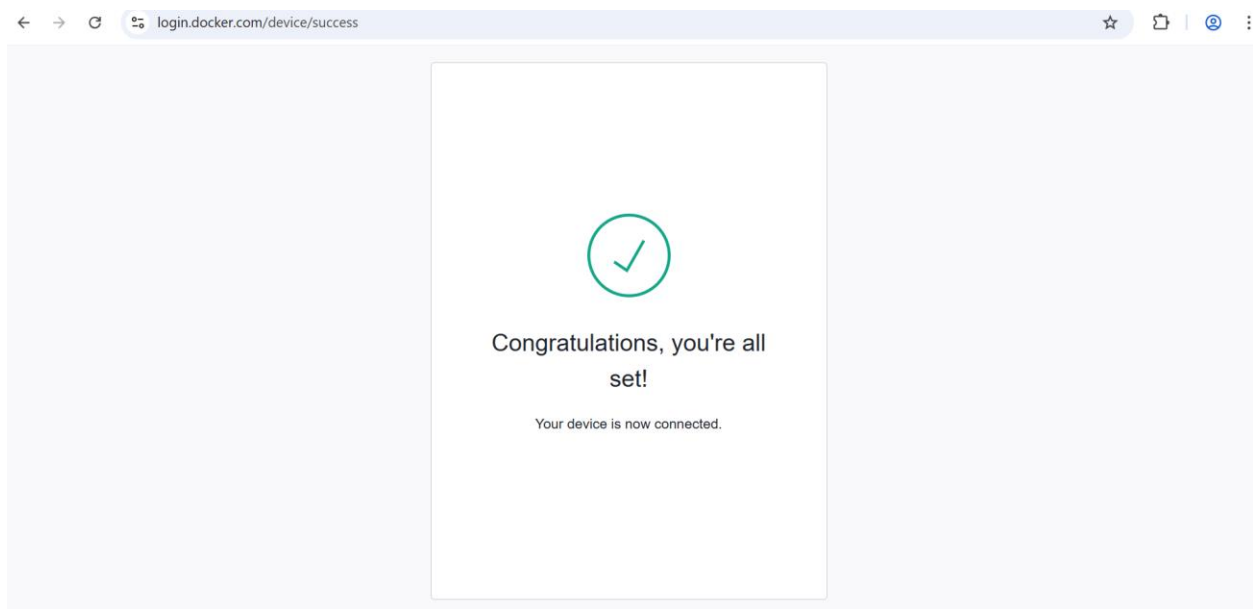
7.3 Create a Docker Hub

Create credentials for docker hub

```
docker login
```

```
ubuntu@ip-172-31-26-69:~$ docker login
USING WEB-BASED LOGIN
Info -> To sign in with credentials on the command line, use 'docker login -u <username>'
Your one-time device confirmation code is: LFGR-FWZ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser...
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
ubuntu@ip-172-31-26-69:~$
```


Login into Docker Hub Via Web based login template (Use one time token Pass)



Check your images and then tag latest image

```
docker tag mydockerhubusername/myubuntuimage:v1 ajitjadhav07/myubuntuimage:v1
```

Verify your images

new tagged image will occur

```
ubuntu@ip-172-31-26-69:~$ docker images
REPOSITORY              TAG          IMAGE ID          CREATED           SIZE
ajitjadhav07/myubuntuimage  v1          57d0fd97704e     17 minutes ago   135MB
mydockerhubusername/myubuntuimage  v1          57d0fd97704e     17 minutes ago   135MB
nginx                    latest       058f4935d1cb     6 days ago       152MB
ubuntu                    latest       c3a134f2ace4     2 months ago     78.1MB
ubuntu@ip-172-31-26-69:~$
```

8. Verify

Run the image to verify its container

```
docker run -it ajitjadhav07/myubuntuimage:v2 bash
```

```
cat /myfile.txt
```

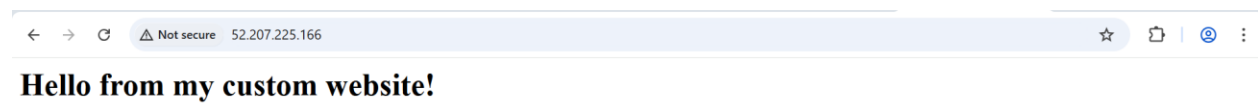
```

root@9ca43blca3ec:/# exit
exit
ubuntu@ip-172-31-26-69:~$ docker images
REPOSITORY              TAG               IMAGE ID           CREATED            SIZE
ajitjadhav07/myubuntuimage v2                8fe98a030187      20 minutes ago    135MB
ajitjadhav07/myubuntuimage v1                57d0fd97704e      About an hour ago  135MB
mydockerhubusername/myubuntuimage v1                57d0fd97704e      About an hour ago  135MB
nginx                    latest            058f4935dicb      6 days ago        152MB
ubuntu                    latest            c3a134f2ace4       2 months ago      78.1MB
ubuntu@ip-172-31-26-69:~$ docker run ajitjadhav07/myubuntuimage:v2
ubuntu@ip-172-31-26-69:~$ docker run -it ajitjadhav07/myubuntuimage:v2 bash
root@ec722444bb99:/# ls/
bash: ls/: No such file or directory
root@ec722444bb99:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  myfile.txt  opt  proc  root  run  sbin  srv  sys  usr  var
root@ec722444bb99:/# cat /myfile.txt
Hello world - this is a custom Docker image
root@ec722444bb99:/#

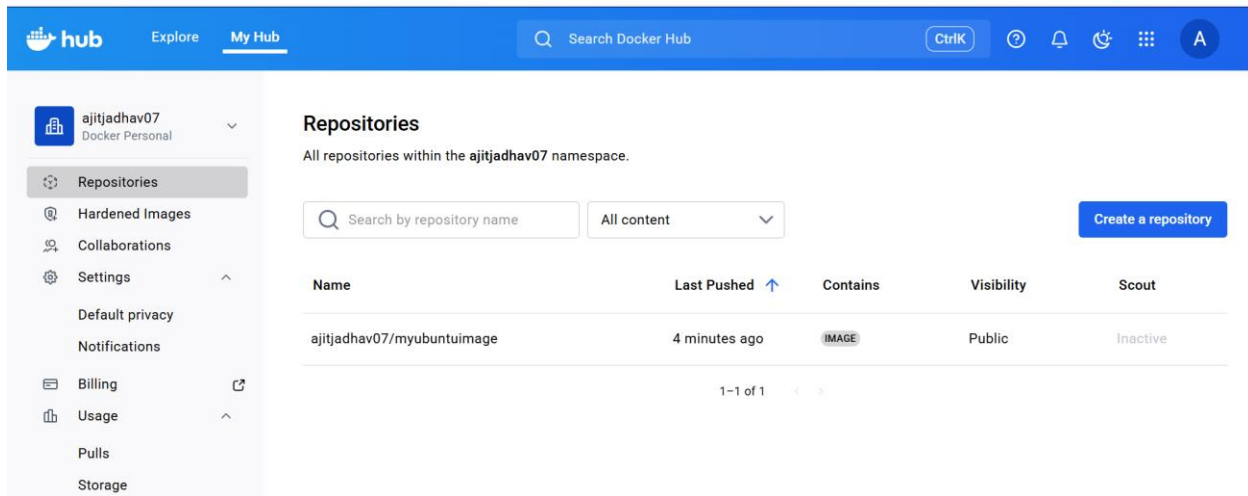
```

Also check Via NGINX on browser

<http://<public-ip>>



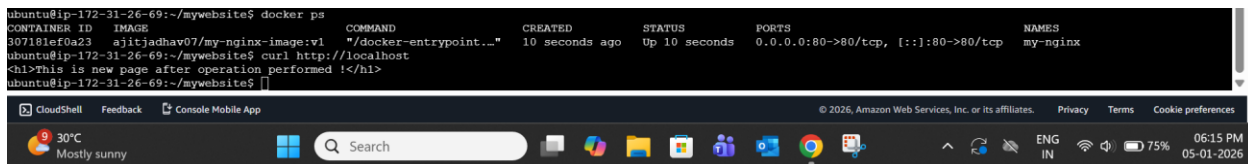
Push it to docker repository



The screenshot shows the Docker Hub interface for a user named 'ajitjadhav07'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main area is titled 'Repositories' and shows a list of repositories within the 'ajitjadhav07' namespace. A search bar and a 'Create a repository' button are at the top. The repository list has columns for Name, Last Pushed, Contains, Visibility, and Scout. One repository is listed: 'ajitjadhav07/myubuntuimage', pushed 4 minutes ago, containing an 'IMAGE', with 'Public' visibility and an 'Inactive' Scout status.

Name	Last Pushed	Contains	Visibility	Scout
ajitjadhav07/myubuntuimage	4 minutes ago	IMAGE	Public	Inactive

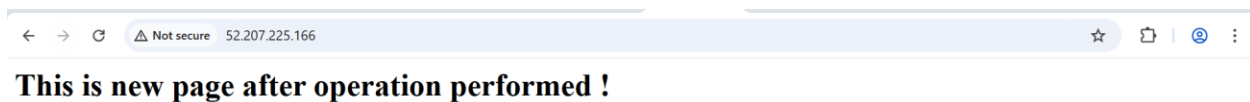
After latest changes done in image, we see result



The screenshot shows a terminal window with the following output:

```
ubuntu@ip-172-31-26-69:~/mywebsite$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
307101ef0a23   ajitjadhav07/my-nginx-image:v1     "/docker-entrypoint..." 10 seconds ago Up 10 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp my-nginx
```

Below the terminal, a web browser window shows a new page after operation performed. The browser's address bar shows 'http://localhost'. The bottom of the screen shows a Windows taskbar with the date and time '06:15 PM 05-01-2026'.



Push to Docker Hub

